本週候選議題：每題兩頁

01 (build operator) • feat • 第 2–3 頁
讓多模型共用 selective operator build

02 (build operator) • feat • 第 4–5 頁
補齊 col2im.out portable kernel

03 (build operator) • feat • 第 6–7 頁
補齊 avg_pool1d.out portable kernel

04 (build operator) • feat • 第 8–9 頁
補齊 avg_pool3d.out portable kernel

05 (build operator) • feat • 第 10–11 頁
補齊 adaptive_avg_pool1d.out

06 (build operator) • feat • 第 12–13 頁
補齊 _adaptive_avg_pool3d.out

07 (build operator) • bug • 第 14–15 頁
將 custom kernel 範例納入 CMake 測試

08 (quantization) • feat • 第 16–17 頁
規劃 Cortex-M Int16 算子支援

09 (quantization) • feat • 第 18–19 頁
規劃 Cortex-M packed int4 權重路徑

10 (quantization) • feat • 第 20–21 頁
續查既有 quantizer summary 修正

1. (build operator) 讓多模型共用 selective operator build

原始議題 ↗

來源更新 2026-03-11

問題釐清

一個 runtime 要跑多個模型，但各模型的算子需求不同。目前作者在外部合併 YAML，缺少方便的多模型輸入或官方建議流程。

可能解法

候選方案：讓模型／YAML 輸入接受清單，再取算子聯集；保留單模型行為。先確認目前 CMake 介面並測試重複算子，尚未實作。

應用場景

例如同一裝置同時部署影像與語音模型：希望共用一份 runtime，並只編入兩個模型實際需要的算子。

專有名詞介紹

Selective build：只編入需要的算子。PTE：ExecuTorch 模型檔。算子聯集：合併多個模型的需求，重複項只保留一次。

1. (build operator) 讓多模型共用 selective operator build

原始議題 ↗

來源更新 2026-03-11

重點功能

一次收集多個模型需要的算子，建立可共用的精簡 runtime。

Input

多個 PTE 或算子 YAML 清單。

例：模型 A 需要 add、relu；模型 B 需要 mul、relu。

Output

合併後的算子集合與相應 runtime。

上述例子只需 add、mul、relu；實際介面待確認。

2. (build operator) 補齊 col2im.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

問題釐清

原票只要求 col2im.out，未列 dtype、layout 或測試範圍。必須先核對現行匯出與算子註冊，確認仍有 portable 支援缺口。

可能解法

候選方案：依 ATen 的 fold 語意補上 kernel 與註冊；先測重疊區域累加、輸出形狀及邊界。此為方案，尚未本機驗證。

應用場景

例如將切成小區塊處理的影像特徵拼回完整特徵圖；匯出圖若保留 col2im，就需要 runtime 能執行它。

專有名詞介紹

col2im／Fold：將區塊還原為影像排列，重疊處相加。Portable kernel：不依賴特定加速器的算子實作。out：寫入指定輸出張量。

2. (build operator) 補齊 col2im.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

重點功能

把區塊形式的資料組回特徵圖，正確累加重疊位置。

Input

影像區塊張量、目標高寬及區塊排列參數。

批次形式可視為 [N, C×區塊面積, 區塊數]。

Output

指定高寬的 [N, C, H, W] 張量。

同一位置有多個區塊貢獻時，輸出為它們的總和。

3. (build operator) 補齊 avg_pool1d.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

問題釐清

原票要求一維平均池化的 portable kernel，沒有 dtype 或形狀驗收細節。需先查匯出是否已透過算子改寫覆蓋此能力。

可能解法

候選方案：比較專用 1D kernel 與既有 2D 路徑改寫；以 ATen 對照視窗、邊界補值與取整行為，確認缺口再開發。

應用場景

例如語音或感測時序模型：沿時間軸平均取樣，縮短序列，讓後續網路處理較少的位置。

專有名詞介紹

Average pooling：對局部視窗取平均。Stride：視窗移動的步長。Decomposition：將算子改寫成其他已支援算子的組合。

3. (build operator) 補齊 avg_pool1d.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

重點功能

沿一維序列計算滑動視窗平均值。

Input

序列張量 [N, C, L] 與視窗大小、步長。

例：資料 [1, 3, 5, 7]，視窗 2、步長 2、不補值。

Output

平均池化後的 [N, C, Lout]。

上述例子輸出 [2, 6]；批次與通道數保持不變。

4. (build operator) 補齊 avg_pool3d.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

問題釐清

原票要求三維平均池化的 portable kernel，規格描述極少。昨天列為 coverage 候選；仍需核對目前實作，不能把開票等同於確認缺失。

可能解法

候選方案：先用小型三維張量核對形狀與平均除數，再補 kernel ／註冊；邊界補值、取整與 dtype 範圍需明確定義。

應用場景

例如影片或體積資料的神經網路：沿深度／時間、高與寬三軸取平均，減少特徵圖的空間尺寸。

專有名詞介紹

3D pooling：在三個空間軸上取局部平均。NCDHW：批次、通道、深度、高、寬。Divisor：計算平均時使用的除數。

4. (build operator) 補齊 avg_pool3d.out portable kernel

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

重點功能

對三維區塊取平均，產生較小的三維特徵圖。

Input

特徵張量 [N, C, D, H, W] 與三軸視窗、步長。

例：[1, 1, 4, 4, 4]，視窗與步長皆為 2，不補值。

Output

池化後的 [N, C, Dout, Hout, Wout]。

上述例子為 [1, 1, 2, 2, 2]；每格是對應區塊的平均值。

5. (build operator) 補齊 adaptive_avg_pool1d.out

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

問題釐清

原票要求 adaptive_avg_pool1d 的 portable 支援，未提供失敗紀錄。需先看匯出圖如何改寫此 op，以及 runtime 最終使用哪個 schema。

可能解法

候選方案：以最小匯出模型確認路徑，再決定 kernel 或算子改寫。依 ATen 測試非整除分段、單點輸出及不同輸出長度。

應用場景

例如將長度不同的語音／時序特徵轉成固定長度，讓分類頭接受一致的特徵尺寸。此為算子用途示例。

專有名詞介紹

Adaptive pooling：指定輸出大小，自動決定取平均的區間。Schema：算子的輸入與輸出介面。L：序列長度。

5. (build operator) 補齊 adaptive_avg_pool1d.out

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

重點功能

把一維特徵轉成指定長度，以各區間平均值作為輸出。

Input

序列張量 [N, C, Lin] 與目標長度 Lout。

例：[1, 3, 5, 7]，目標長度為 2。

Output

固定長度的 [N, C, Lout]。

上述例子為 [2, 6]；不必手動指定視窗與步長。

6. (build operator) 補齊 _adaptive_avg_pool3d.out

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

問題釐清

原票要求三維 adaptive average pooling 的 portable kernel，但描述極少。先查現行匯出與註冊，確認需要補上的實際算子路徑。

可能解法

候選方案：依 ATen 的三軸分段規則測試非整除區間，再補形狀計算、累加及註冊；先確認正確性，尚未執行驗證。

應用場景

例如不同長度影片或不同大小的體積特徵，在接分類頭前統一為固定的深度、高與寬。此為算子用途示例。

專有名詞介紹

Adaptive 3D pooling：把三軸特徵平均成指定大小。D／H／W：深度、高、寬。ATen：PyTorch 的張量算子層。

6. (build operator) 補齊 _adaptive_avg_pool3d.out

原始議題 ↗ 算子語意 ↗

來源更新 2025-06-03

重點功能

將三維特徵平均成指定的輸出深度、高與寬。

Input

三維特徵 [N, C, Din, Hin, Win] 與目標大小。

例：[1, 8, 4, 6, 6]，目標為 [1, 1, 1]。

Output

[N, C, Dout, Hout, Wout]。

上述例子為 [1, 8, 1, 1, 1]；每個通道得到一個全域平均值。

7. (build operator) 將 custom kernel 範例納入 CMake 測試

原始議題 ↗ 相關來源 ↗

來源更新 2025-01-28

問題釐清

原文回報 custom_kernel_example 只有 buck2 建置設定，未納入 CMake 或 OSS CI。這是範例的建置／測試覆蓋缺口，並非已證實的算子數值錯誤。

可能解法

候選方案：確認範例現況，新增合適的 CMake target 與測試入口，讓公開 CI 實際編譯及執行。既有修正連結保留供比對。

應用場景

以 CMake 開發自訂算子的使用者，需要可執行的範例；維護者也需要公開 CI 及早攔下範例的建置或註冊回歸。

專有名詞介紹

Custom kernel：自訂算子實作。CMake／buck2：兩套建置工具。OSS CI：開源專案的自動建置與測試流程。

7. (build operator) 將 custom kernel 範例納入 CMake 測試

原始議題 ↗ 相關來源 ↗

來源更新 2025-01-28

修正前

依原始回報，範例只有 buck2 設定；CMake 與 OSS CI 沒有對應覆蓋。公開測試無法藉此範例攔下建置或註冊回歸。

修正後（預期）

預期同一範例可由 CMake 建置，並在 OSS CI 實際執行；連結、註冊或執行失敗時，對應測試應失敗。這是驗收需求，尚未確認修復。

8. (quantization) 規劃 Cortex-M Int16 算子支援

原始議題 ↗ 相關來源 ↗

來源更新 2025-12-02

問題釐清

需求是讓 CMSIS-NN 已支援的算子走 int16 路徑，涉及 quantizer、passes 與 C++ kernel。依賴前置量化設定議題，不是獨立的一週小修。

可能解法

候選方案：先確認前置設定介面，選一個算子做端到端切片；對照 reference 數值後，再擴充其他 CMSIS-NN 已支援的算子。

應用場景

在 Cortex-M 微控制器部署模型時，若要評估 int16 量化數值表現，就需要匯出流程與目標 kernel 都接受該配置。改善幅度須實測。

專有名詞介紹

Cortex-M：微控制器處理器系列。CMSIS-NN：其神經網路核心庫。Int16：16 位整數表示。Quantizer：指定量化策略的元件。

8. (quantization) 規劃 Cortex-M Int16 算子支援

原始議題 ↗ 相關來源 ↗

來源更新 2025-12-02

重點功能

讓 CMSIS-NN 支援的算子可用 int16 量化路徑執行。

Input

模型、int16 量化配置與支援的算子集合。

前置量化設定介面及各算子的 dtype 契約仍待確認。

Output

可在目標 Cortex-M 執行的 int16 算子路徑／模型。

結果需對照參考實作；非所有算子一律支援。

9. (quantization) 規劃 Cortex-M packed int4 權重路徑

原始議題 ↗ 相關來源 ↗

來源更新 2025-12-02

問題釐清

原文要求 CMSIS-NN 已支援算子的 packed int4 路徑，涵蓋 quantizer、kernel 與 lowering。依賴前置量化配置，可能還需要權重重排。

可能解法

候選方案：先挑單一算子確認排列、正負號與補值契約；驗證 pack／unpack，再接 lowering 與目標 backend 的數值測試。

應用場景

Cortex-M 裝置的模型權重空間有限時，可評估以 4-bit 權重壓縮儲存；實際總大小與效能仍取決於排列、補值及目標算子。

專有名詞介紹

Int4：4 位整數。Packed：在一個 byte 放入兩個 4-bit 值。Lowering：把模型算子轉成 backend 可執行形式。

9. (quantization) 規劃 Cortex-M packed int4 權重路徑

原始議題 ↗ 相關來源 ↗

來源更新 2025-12-02

重點功能

讓支援的算子使用 packed int4 權重，接通量化與執行路徑。

Input

模型權重、int4 量化配置與目標算子。位元排列及重排方式須符合 CMSIS-NN 的個別算子契約。

Output

供目標 kernel 使用的 packed 權重與可執行模型。

一般儲存單位為每 byte 兩個 int4；數值正確性需目標驗證。

10. (quantization) 續查既有 quantizer summary 修正

原始議題 ↗ 相關來源 ↗

來源更新 2025-09-08

問題釐清

量化流程可能顯示成功，實際卻沒有量化任何節點。昨天已記錄個人 fork PR 14；本題是續查既有工作，不代表已重新選定或要重寫。

可能解法

候選方案：先比較既有 fork 與 main，區分 annotation 與最終量化統計；補清楚零命中警告、模組分組與選擇性量化的行為。

應用場景

切換量化策略或只量化部分模組時，使用者希望快速看出哪些算子被處理，以及是否不小心全部未命中。

專有名詞介紹

Quantizer：安排量化的元件。Annotation：標記預計量化的節點。Summary：彙整種類、數量與設定；不能把標記數冒充最終量化數。

10. (quantization) 續查既有 quantizer summary 修正

原始議題 ↗ 相關來源 ↗

來源更新 2025-09-08

重點功能

顯示量化的算子種類、數量與關鍵設定；零命中時提出警告。

Input

模型圖、量化配置與流程中的量化資訊。

統計取自 annotation 或 conversion，需在介面中明確標示。

Output

精簡摘要與必要警告。

例：Linear × 10（4-bit、groupwise）；沒有任何命中時明確提醒。